

Python file handling basics

Thought: Identified straightforward knowledge question requiring direct instruction.

File Handling in Python

Python makes file handling straightforward with built-in functions. Here's a complete guide:

1. Opening & Closing Files

```
# Basic open/close
file = open("example.txt", "r")
file.close() # Always close!

# ✅ Best practice: use 'with' (auto-closes)
with open("example.txt", "r") as file:
    content = file.read()
```

File Modes

Mode	Description
"r"	Read (default)
"w"	Write (overwrites file)
"a"	Append (adds to end)
"x"	Create (fails if exists)
"rb" / "wb"	Read/Write binary

2. Reading Files

```
with open("example.txt", "r") as f:

    content = f.read()          # Entire file as one string

    first_100 = f.read(100)     # Read 100 characters

    line = f.readline()        # One line at a time

    lines = f.readlines()      # List of all lines

# ✅ Most memory-efficient – iterate line by line
with open("example.txt", "r") as f:
    for line in f:
        print(line.strip())    # .strip() removes \n
```

3. Writing Files

```
# Write mode – creates file or overwrites existing
with open("output.txt", "w") as f:
    f.write("Hello, World!\n")
    f.write("Second line\n")

# Append mode – adds to existing content
with open("output.txt", "a") as f:
    f.write("This is appended\n")

# Write multiple lines at once
lines = ["line 1\n", "line 2\n", "line 3\n"]
with open("output.txt", "w") as f:
    f.writelines(lines)
```

4. Working with File Paths (`os` & `pathlib`)

```
import os
from pathlib import Path

# Check if file exists
os.path.exists("file.txt")          # True / False
Path("file.txt").exists()           # Same, modern way

# Join paths safely (works on all OS)
path = os.path.join("folder", "sub", "file.txt")
path = Path("folder") / "sub" / "file.txt" # Modern

# Get file info
os.path.getsize("file.txt")         # Size in bytes
os.path.basename("/dir/file.txt")   # → "file.txt"
os.path.dirname("/dir/file.txt")    # → "/dir"

# List directory contents
os.listdir(".")                     # Files in current dir
```

5. Handling Exceptions

```
try:
    with open("missing.txt", "r") as f:
        content = f.read()
except FileNotFoundError:
    print("File does not exist!")
except PermissionError:
    print("No permission to read this file!")
```

```
except IOError as e:
    print(f"IO error: {e}")
```

6. Working with CSV Files

```
import csv

# Reading CSV
with open("data.csv", "r") as f:
    reader = csv.DictReader(f)    # Rows as dicts
    for row in reader:
        print(row["name"], row["age"])

# Writing CSV
data = [{"name": "Alice", "age": 30}, {"name": "Bob", "age": 25}]
with open("data.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "age"])
    writer.writeheader()
    writer.writerows(data)
```

7. Working with JSON Files

```
import json

# Read JSON
with open("data.json", "r") as f:
    data = json.load(f)    # Parses into dict/list

# Write JSON
data = {"name": "Alice", "scores": [95, 87, 92]}
```

```
with open("data.json", "w") as f:
    json.dump(data, f, indent=4)  # indent for pretty print
```

8. Useful File Operations

```
import os
import shutil

os.rename("old.txt", "new.txt")           # Rename
os.remove("file.txt")                     # Delete file
os.mkdir("new_folder")                    # Create directory
os.makedirs("a/b/c", exist_ok=True)       # Create nested dirs
shutil.copy("src.txt", "dst.txt")         # Copy file
shutil.move("src.txt", "folder/")         # Move file
```

Quick Reference

Read whole file	→ <code>f.read()</code>
Read line by line	→ <code>for line in f</code>
Write new file	→ <code>open("f", "w")</code>
Add to file	→ <code>open("f", "a")</code>
Safe path joining	→ <code>Path("dir") / "file"</code>
Structured data	→ <code>csv</code> / <code>json</code> modules
Always use	→ <code>with open(...) as f</code>

The `with` statement is the golden rule — it guarantees the file is properly closed even if an error occurs. Want me to go deeper on any specific topic, like binary files, file searching, or async file I/O?